ADR 0002: API style (GraphQL for the app, REST for everything else), MVP on REST

Status	**Proposed**. The meeting deferred "REST or GraphQL" to the next technical meeting
Date	2026-10-19
Related	`05-api-design.md`, `06-mvp-slice-and-migration.md`, `adr/0007`, `adr/0008` (the vendor, admin and vendor-application REST lives in the FastAPI vendor-admin service; the Razorpay webhook is in Core; this ADR decides only the customer app API on Core)

Context

The mobile app's screens need different slices of the same data (a product card, a product page, a cart line, a closet with members). The vendor portal, website forms and webhooks are simple resource calls or fixed-format callbacks. The scaffold already uses GraphQL for the app and REST for the vendor portal, public site endpoints and webhooks. Today's app has about 40 direct `fetch` calls with no shared client, so switching the app's protocol means rewriting its networking layer. The MVP deadline leaves no room for that.

Options

1. **REST everywhere** (OpenAPI, generated client). One style, simplest for the current app and the deadline.
2. **Hybrid:** GraphQL for the app, REST for vendor portal, public site and webhooks (the scaffold).
3. **GraphQL everywhere.** Webhooks and uploads fit poorly.

Decision (proposed)

Option 2 as the **target**, with the **MVP slice on REST**:

- Until cutover, the app uses today's REST routes under `/v1`, with the changes in `06` (tokens, no vectors).
- The public, order and admin routes added for the MVP use the same resource names as the target REST design.
- GraphQL production safeguards: persisted queries, depth limit 10, alias limit 15, no introspection or IDE outside development.
- The GraphQL schema file and the OpenAPI file are committed; CI blocks stale files and breaking changes.

Consequences

- The app needs a one-time networking rewrite at cutover, behind a remote flag with fallback to REST.
- Two documented contracts to maintain.
- If the team decides the cutover cost is not worth it, the same operations are exposed as REST and the GraphQL layer is dropped; the services layer does not change.

Revisit when

The app team has capacity for the rewrite, a web storefront needs different data shapes, or the next technical meeting decides otherwise.